

第 15 章 一次性任务、周期任务与 systemd Timer

从“什么时候运行”走到“以谁、在什么环境、怎样留下可验证结果”。

对象模型 操作语义 验证 诊断 经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：计划任务不是一条“到点运行”的孤立命令，而是一条由调度定义、调度器、执行身份、执行环境、工作负载和结果证据共同组成的链路。

专题地图

- 知识专题 先识别任务生命周期，再选择调度器
- 操作专题 使用 `at` 安排未来的一次执行
- 知识专题 正确解释 cron 五字段与日期匹配
- 操作专题 用户 crontab、系统 crontab 与 `/etc/cron.d`
- 诊断专题 手工执行成功，但 cron 失败
- 知识专题 timer 和 service 是两个状态对象
- 操作专题 持久 timer、transient timer 与安全迁移
- 诊断专题 timer 在等待，但任务没有正确完成
- 经典任务 周期健康检查与 cron→timer 迁移

阅读时持续回答

1. 这是一次性、固定日历还是相对时间任务？
2. 调度定义保存在哪里，由哪个进程解释？
3. 任务最终以哪个用户和工作目录执行？
4. `PATH`、`SHELL`、`HOME` 与输出是否明确？
5. 当前证据只证明“已配置”，还是已经证明“已触发”？
6. timer 状态与 service 结果是否分别检查？
7. 关机错过后是否需要补执行？
8. 最终产物是否正确、非空且时间合理？

01 识别生命周期一次、日历重复或相对时间

02 确认定义 job、crontab 或 timer

03 确认调度器 atd、crond 或 systemd

04 重建执行环境用户、目录、PATH 与输出

05 判断触发与退出 NEXT/LAST、Result 与退出码

06 验收真实产物内容、所有者、大小和时间

阅读提示：先用概念块建立对象边界，再用操作语义速查确认接口，最后通过专题、经典任务和参考解答完成分层验证。

第 15 章 · 正文

计划任务表面上是在回答“什么时候运行”，真正决定任务是否可靠的却不止时间。调度定义必须交给某个调度器；调度器到点后以特定用户、环境和工作目录启动命令；命令还要把退出状态、标准输出、标准错误和业务产物留成证据。只看到一行 crontab、一个 at job，或者一个处于 `active (waiting)` 的 timer，都不能证明任务已经正确完成。本章从三类需求切入：未来只执行一次、按日历反复执行、由 systemd 按日历或相对时间激活。学习重点不是把三套语法混在一起背，而是建立一条统一证据链：

调度需求

- 调度定义
- 调度器状态
- 执行身份与环境
- 工作负载退出状态
- 输出、日志与真实产物

最常见的误判有三类：手工执行成功就认为 cron 一定成功；把用户 crontab 与系统 cron 的字段混为一谈；看到 timer 正在等待就认为 service 已经正确完成。systemd 的通用 unit 语义属于第 12 章，日志系统属于第 13 章，时区与同步属于第 14 章。本章只引用这些前置证据，不重新展开它们。

概念

一次性任务 是只在未来某个时刻取出并执行一次的调度对象。本章以 `at` job 为代表：提交后它进入由 `atd` 管理的队列，被执行或删除后不会自动生成下一次任务。队列中存在只能证明定义仍在等待，不能证明届时环境、权限和外部依赖一定满足。

概念

周期任务 是反复接受日历匹配的规则，而不是从创建时刻开始不断累加固定间隔。cron 通过分钟、小时、月中日、月份和星期五个字段决定匹配；systemd timer 既能表达日历，也能表达开机后或上次激活后的相对时间。选择对象时应先判断任务生命周期，而不是先选自己最熟悉的命令。

概念

执行身份 决定任务拥有哪些 UID、GID、HOME、文件权限和安全上下文。用户 crontab 天然以该用户运行；系统 crontab 与 `/etc/cron.d` 通过额外用户字段指定身份；timer 通常把身份写在被激活的 service 中。身份错误时，时间表达式完全正确也可能只得到权限失败。

概念

执行环境 是调度器启动工作负载时提供的变量、Shell、工作目录、标准输入输出和可用凭据。它通常比交互登录会话精简，不会自动包含 alias、函数、SSH agent 或终端提示。手工成功而 cron 失败时，最有区分度的证据不是反复重跑，而是以目标用户和 `env -i` 重建精简环境。

概念

日历表达式 描述哪些墙上时刻符合触发条件。cron 使用五字段，月中日与星期在同时受限时具有特殊的 OR 匹配；`OnCalendar=` 使用 systemd 的日历语法，可先由 `systemd-analyze calendar` 展开。语法可解析只证明表达式成立，不证明调度器已加载或工作负载会成功。

概念

timer/service 对 把“何时触发”和“具体执行”拆成两个 unit。`.timer` 管理 NEXT、LAST 和等待状态；`.service` 管理 `User=`、`WorkingDirectory=`、`ExecStart=`、退出码和 journal。timer 的 `active (waiting)` 不能替代 service 的 `Result=`，service 成功也不能替代业务产物检查。

概念

补执行 是系统在日历触发点处于关闭或 timer 未激活时，恢复后补一次遗漏工作的能力。`Persistent=true` 只对日历 timer 有意义，它不是逐条回放所有错过的周期，也不能修复工作负载本身的失败。是否真正补跑必须在可控的关机、停用与恢复场景中实测。

操作语义以下入口分别观察或改变一次性队列、cron 规则与 systemd 调度。先明确命令的作用对象，再选择参数和验证层次。

at 命令组

SYNOPSIS

```
at [options] TIMESPEC
atq
at -c JOB_ID
atrm JOB_ID
```

提交、列出、审阅和删除由 `atd` 管理的一次性任务。

重要参数 / 形式

`at now + 20 minutes`

从标准输入读取命令，并安排在 20 分钟后执行一次。

`-f FILE`

从文件读取任务正文，适合需要保存和审阅的作业。

`at -c JOB_ID`

查看保存的完整脚本、环境、umask、工作目录与最终命令。

`atrm JOB_ID`

删除尚未执行的队列项；不能停止已经启动的进程。

crontab

SYNOPSIS

```
crontab [-u USER] {-e|-l|-r}
crontab [-u USER] FILE
```

编辑、列出、删除或安装某个用户的 crontab。用户表每条任务只有五个时间字段，执行用户由表的所有者决定。

重要参数 / 形式

`-e`

通过编辑器修改目标用户的 crontab。

`-l`

列出当前安装的任务表，是变更前后最直接的静态证据。

`-u USER`

由 root 管理指定用户的表，避免误查当前登录用户。

`-r`

删除整张表，不是删除一行；使用前必须明确风险。

系统 crontab 与 `/etc/cron.d`

SYNOPSIS

```
minute hour day month weekday user command
```

由系统管理员集中保存任务，并在五个时间字段之后显式指定执行用户。

重要字段 / 形式

`user`

系统表额外字段；写进用户 crontab 会把用户名当成命令。

`PATH=...`

显式限制命令搜索范围，避免依赖登录环境。

`SHELL=/bin/bash`

指定解释任务行的 Shell；它不自动加载用户的交互启动文件。

`MAILTO=""`

禁用 cron 邮件尝试；重要任务仍应显式保存 stdout/stderr。

`\%`

在命令部分表示字面百分号；未转义 ``%`` 会切分命令与标准输入。

systemd timer

SYNOPSIS

```
[Timer]
OnCalendar=...
OnBootSec=...
OnUnitActiveSec=...
Persistent=true
```

以 `.timer`` 表达触发时间，以 `.service`` 表达执行身份、命令和退出结果。

重要参数 / 形式

`OnCalendar=`

使用墙上日历时间；可与 ``Persistent=true`` 组合补一次遗漏。

`OnBootSec=`

从系统启动后经过指定时长触发。

`OnUnitActiveSec=`

从目标 unit 上次激活后计算下一次间隔。

`RandomizedDelaySec=`

在允许范围内随机推迟，帮助多台主机摊开负载。

AccuracySec=

允许 systemd 合并唤醒的精度窗口，不是随机延迟。

systemctl list-timers

SYNOPSIS

```
systemctl list-timers [--all] [PATTERN...]
```

列出 timer 的下一一次和上一次触发时间，以及它将激活的 unit。

重要列 / 形式

NEXT / LEFT

预计的下一一次触发时刻和剩余时间。

LAST / PASSED

最近一次触发时刻和距今时间；不表示 service 一定成功。

--all

同时显示未运行或没有下一一次触发时间的 timer，适合排错。

systemd-analyze calendar

SYNOPSIS

```
systemd-analyze calendar EXPRESSION
```

解析 `OnCalendar=` 表达式，显示规范形式和后续触发时间。

重要输出 / 边界

Normalized form

展示 systemd 实际理解的规范化表达式。

Next elapse

展示下一一次日历匹配；仍未包含 service 运行结果。

systemd-analyze verify

检查 unit 文件语法和部分引用关系，不能替代真实执行。

systemd-run

SYNOPSIS

```
systemd-run --on-active=TIME COMMAND [ARG...]  
systemd-run --on-calendar=EXPR COMMAND [ARG...]
```

创建 transient timer 与 service，用于短期验证 systemd 调度，不在磁盘上建立持久 unit 文件。

重要参数 / 形式

`--on-active=`

从 transient timer 激活后计算延迟。

`--on-calendar=`

使用 systemd 日历表达式创建临时日历 timer。

`--unit=NAME`

指定便于查询的 unit 基名，避免依赖自动生成名称。

`--property=...`

为 transient service 设置受支持的 unit 属性；使用前应核对本机版本。

[知识专题] 先识别任务生命周期，再选择调度器

一次性、周期日历和 systemd 事件调度解决的不是同一个问题。最短的命令未必是最稳的方案，功能更多的 timer 也不自动优于 cron。考试题如果明确要求 crontab，应按题意使用 cron；已有简单而稳定的 cron 任务，也不应只为了“看起来现代”而迁移。

① [知识点] 三类调度对象的边界

需求	首选对象	定义保存在哪里	最关键的验收
将来执行一次	<code>at job</code>	at spool, 由 <code>atd</code> 管理	队列、任务正文、实际产物
简单重复日历	crontab entry	用户 crontab、 <code>/etc/crontab</code> 或 <code>/etc/cron.d/*</code>	字段、用户、crond、输出和产物
日历或相对时间，并要求补跑、随机延迟、依赖和统一结果	<code>.timer + .service</code>	systemd unit 文件	NEXT/LAST、service Result、journal 和产物
临时验证一次 systemd 调度思路	transient timer	systemd manager 内存状态	生成的 timer/service、结果和非持久边界

`at` 适合“今晚执行一次”或“20 分钟后恢复配置”。`cron` 适合“每周三 15:30”这类稳定日历。`systemd timer` 适合“系统关机错过后补一次”“开机 10 分钟后执行”“每次任务激活后再过 30 分钟”“多台主机随机摊开”等需求。

② [知识点] 调度定义和工作负载是两个对象

调度配置最好只表达：

何时
→ 以谁
→ 启动哪个稳定入口

业务逻辑应放在可独立运行的脚本或程序中。脚本应有明确 shebang、绝对路径、可执行权限、稳定退出码，以及可检查的输出或产物。把多层管道、条件判断、日期拼接和密钥读取全部塞进一行 crontab，会让

引用、`%`、工作目录和错误处理同时变得脆弱。

对 `systemd timer`，这个边界更加明确：`timer` 不直接持有 `ExecStart=`，它激活 `service`；`service` 才定义 `User=`、`WorkingDirectory=`、`EnvironmentFile=` 和 `ExecStart=` 等执行属性。

③ [知识点] 任务至少有五个状态层次

定义状态：任务是否存在、语法和目标是否正确
调度状态：调度器是否运行，任务是否正在等待
触发状态：是否到点并发生过触发
执行状态：命令以什么身份退出，退出码是什么
功能状态：日志、报告、文件或远端动作是否符合目标

典型误判包括：

- `atq` 中有任务，于是声称未来一定成功；
- `crond.service` 为 `active`，于是声称某一条 `cron job` 已成功；
- `timer` 为 `active (waiting)`，于是声称同名 `service` 最近执行成功；
- `service` 显示 `Result=success`，却没有检查脚本是否把错误吞掉或写错目录；
- 报告文件存在，却没有检查所有者、大小和更新时间。

④ [知识点] 执行身份、环境、目录与输出必须显式化

计划任务通常没有当前终端，也不会自动继承登录会话中的 `alias`、`Shell` 函数、`SSH agent`、图形会话和交互口令。稳定任务至少要回答：

- 以哪个用户和组运行；
- `PATH` 中允许找到哪些程序；
- `HOME` 和工作目录是什么；
- 需要哪些环境变量或凭据；
- `stdout/stderr` 到哪里；
- 失败怎样被发现；
- 是否允许上一次尚未结束时再启动一次；
- 关机错过后是否补跑。

[Cheatsheet] 一次用 `at`，简单日历用 `cron`，需要 `service` 结果、补跑或随机延迟时用 `timer`；先让脚本独立成功，再连接调度器；配置存在不等于任务完成。

[操作专题] 使用 `at` 安排未来的一次执行

`at` 从标准输入或文件读取命令，把它们保存为一次性作业，并由 `atd` 在指定时间执行。提交成功时通常会返回 `job ID` 和计划时间。这个结果只证明任务进入队列；实际执行仍依赖 `atd`、访问控制、保存的环境、目录权限和工作负载本身。

① [操作] 提交交互式或可审阅的一次性任务

作用对象：当前用户提交给 `atd` 的一次性 Shell 作业。基本语义：`at TIMESPEC` 从标准输入读取命令；交互模式用 `Ctrl+D` 结束。`-f FILE` 从文件读取。典型形式：

```
at now + 20 minutes
at 22:30 tomorrow
at -f /root/jobs/collect-report.at 23:00
```

交互输入适合很短的任务；需要审计时，here-document 更容易回看：

```
at now + 20 minutes <<'ATJOB'
/usr/local/sbin/collect-report \
  >> /var/log/collect-report.log 2>&1
ATJOB
```

引用结束标记 `'ATJOB'` 可以阻止当前 Shell 在提交时展开 `$` 变量、命令替换和反斜杠。这样，变量会在未来执行时由保存的脚本处理。若确实要把当前值固定进任务，应明确取消引用，并在 `at -c JOB` 中确认最终正文。

② [知识点] `at` 保存环境，但不等于重建交互会话

`at` 会生成一段将来交给 Shell 的脚本，其中通常包含提交时的大量环境信息、`umask` 和工作目录切换。它不会保留一个可交互终端，也不能可靠依赖 `alias`、Shell 函数、口令提示或临时挂载。

因此任务正文仍应：

- 使用绝对命令路径；
- 明确 `stdout/stderr`；
- 避免需要输入；
- 检查目标路径在将来仍存在；
- 对网络、挂载和凭据等外部依赖保留失败证据。

③ [查询] 用 `atq` 和 `at -c` 审核队列

```
atq
at -c 12
at -c 12 | tail -n 35
```

`atq` 通常显示 job ID、计划时间、队列和所有者。`root` 可以看到更多用户的作业；普通用户只管理自己的队列。`at -c JOB` 输出保存的完整脚本，内容可能很长，审核重点是：

```
执行用户
→ 保存的 PATH/HOME
→ 工作目录切换
→ umask
→ 最后的命令正文和重定向
```

`at -c` 是提交后的静态证据，它不能证明未来时刻依赖仍然满足。

④ [操作] 用 `atrm` 删除尚未执行的作业

```
atrm 12
# 等价入口在实现支持时也可见: at -d 12
atq
```

删除后应再次检查队列。job ID 消失只证明队列项被取消；如果作业已经开始执行，删除队列项不能代替停止正在运行的进程。进程控制属于第 11 章《进程、作业、信号与调度优先级》。

⑤ [验证点] 分层验证 `at` 作业

```
systemctl is-active atd.service
systemctl is-enabled atd.service
atq
at -c 12 | tail -n 35
journalctl -u atd.service --since today --no-pager
stat /var/log/collect-report.log
```

验收时至少回答：

1. `atd` 当前是否运行；
2. 任务是否仍在队列，时间和所有者是否正确；
3. `at -c` 中的正文、目录和环境是否符合预期；
4. 到点后队列项是否被取走；
5. `daemon` 日志是否显示执行活动；
6. 目标日志或业务产物是否存在、非空且更新时间合理。

⑥ [边界] 访问控制和邮件不能替代产物验证

`/etc/at.allow` 和 `/etc/at.deny` 控制哪些用户可提交作业。常见逻辑是：存在 `allow` 时只允许其中用户；不存在 `allow` 而存在 `deny` 时允许未被拒绝的用户；两者都不存在时行为应以本机 `at(1)` 和 PAM 配置为准。

`at` 可以发送输出邮件，但这依赖本地邮件传输能力。没有收到邮件，既可能表示没有输出，也可能表示邮件链不可用。重要任务应显式重定向并保留真实产物。

[Cheatsheet] `at TIME` 提交；`atq` 查队列；`at -c JOB` 查保存脚本；`atrm JOB` 取消；最终还要看 `atd`、日志和产物。

[知识专题] 正确解释 cron 五字段与日期匹配

cron 每分钟检查一次日历字段是否匹配。字段顺序本身不复杂，高频错误来自把“月中日”和“星期”当作必须同时匹配，或者把系统 crontab 的用户字段误写进用户 crontab。应先确定配置入口，再解释字段。

① [知识点] 用户 crontab 的五字段

分钟 小时 月中日 月 星期 命令

允许值的核心范围：

字段	常用范围
分钟	0-59
小时	0-23
月中日	1-31
月	1-12 或实现支持的英文名称
星期	0-7，0 和 7 通常都表示星期日；也可使用英文名称

例：

30 15 * * 3 /usr/local/libexec/weekly-check

表示每周三 15:30，而不是“每月 3 日”。

② [知识点] *、列表、范围与步长

形式	语义	示例
*	该字段所有允许值	* * * * * 每分钟匹配
1,15,30	指定列表	分钟字段在 1、15、30 分匹配
1-5	含端点范围	星期一到星期五
*/10	从字段起点按步长匹配	每 10 分钟
8-18/2	范围内按步长匹配	8、10、12、14、16、18 时

*/10 不是“从创建时刻开始每 10 分钟”，而是对日历字段做整除式匹配。需要从上一次激活后计算间隔时，考虑 OnUnitActiveSec=。

③ [知识点] 月中日与星期是 OR，不是普通 AND

cron 要求分钟、小时和月份匹配；对于两个“日”字段，只要月中日或星期至少一个匹配即可。当两个字段都被限制时，这会产生 OR 行为。

```
0 6 1 * 1 /usr/local/libexec/report
```

它不是“仅在每月 1 日恰好为星期一时运行”，而是通常在每月 1 日以及每个星期一 06:00 运行。需要严格 AND 时，应使用脚本内条件、拆分规则，或用 `OnCalendar=` 表达并通过 `systemd-analyze calendar` 展开验证。

④ [知识点] 日历异常不能只靠语法解释

夏令时转换可能出现不存在的本地时间或重复的本地时间，导致 cron job 漏跑或重复运行。时区和系统时间配置属于第 14 章《系统时间、时区、RTC 与 chrony》，本章只建立边界：

- 调度表达式按系统采用的时间基准解释；
- 修改时区后，应重新核对 cron 与 timer；
- 对不能漏跑的日历任务，timer 的 `Persistent=` 可能更合适，但仍需验证实际语义；
- 对不能重复执行的任务，工作负载本身应具备幂等或锁机制。

[Cheatsheet] 用户 crontab 是五个时间字段加命令；`*`，`-` / 分别表示全部、列表、范围和步长；分钟、小时、月匹配，并且月中日或星期至少一个匹配。

[操作专题] 管理用户 crontab、系统 crontab 与 `/etc/cron.d`

配置入口决定字段格式和执行身份。用户 crontab 天然归属于某个用户，不写用户字段；`/etc/crontab` 与 `/etc/cron.d/*` 由管理员维护，每一条任务必须显式写执行用户。

① [操作] 管理用户 crontab

```
crontab -l
crontab -e
crontab -l -u monitor
crontab -u monitor -e
```

- `crontab -e` 在临时文件中编辑并安装新表；
- `crontab -l` 列出已安装表；
- root 使用 `-u USER` 管理其他用户；
- `crontab -r` 删除整张表，不是删除一行。

删除或批量替换前先保存基线：

```
crontab -l -u monitor > /root/monitor.crontab.before
```

不要直接编辑 `/var/spool/cron` 中的内部文件。它们的路径、标签和管理方式属于实现细节，应通过 `crontab` 命令维护。

② [知识点] 用户表和系统表的用户字段

用户 `crontab`:

```
分钟 小时 月中日 月 星期 命令
```

`/etc/crontab` 和 `/etc/cron.d/*`:

```
分钟 小时 月中日 月 星期 执行用户 命令
```

典型错误:

```
# 错误: 这是 monitor 的用户 crontab, 却额外写了 monitor
30 15 * * 3 monitor /usr/local/libexec/weekly-check
```

`cron` 会把 `monitor` 当作命令名或命令正文的一部分, 而不是用户字段。

```
# 错误: 这是 /etc/cron.d/weekly-check, 却漏掉执行用户
30 15 * * 3 /usr/local/libexec/weekly-check
```

字段会整体错位, 任务不会以预期形式执行。

③ [配置] 显式设置 `SHELL`、`PATH` 与 `MAILTO`

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
MAILTO=""

30 15 * * 3 /usr/local/libexec/weekly-check \
    >> /var/log/weekly-check.log 2>&1
```

- `SHELL=` 决定 `cron` 用哪个 Shell 解释命令字段; 它不自动改变脚本自己的 shebang;
- `PATH=` 应包含任务真正需要的目录, 但不要无边界复制整个交互环境;
- `MAILTO=` 指定输出邮件接收者; 空值通常关闭邮件; 邮件能否送达仍依赖 MTA;
- `HOME` 和 `LOGNAME` 通常来自目标用户, 但任务不应依赖模糊的当前目录。

需要固定目录时, 优先让脚本自行定位资源, 或在命令中显式进入目录:

```
0 2 * * * cd /srv/report && /usr/local/libexec/build-report \
>> /var/log/build-report.log 2>&1
```

若 `cd` 失败, `&&` 阻止后续命令在错误目录运行。

④ [知识点] 未转义 `%` 会改变命令和标准输入

在 `crontab` 命令字段中, 第一个未转义 `%` 会被转换为换行; 前半部分作为命令, 后半部分作为标准输入。其后的未转义 `%` 也转换为换行。

```
# 容易失败: %F 会被 cron 解释
0 1 * * * /usr/bin/date +%F >> /var/log/date.log 2>&1

# 必须直接写在 crontab 时, 应转义
0 1 * * * /usr/bin/date +\%F >> /var/log/date.log 2>&1
```

更稳定的设计是把 `date +%F` 放进脚本, 让 `crontab` 只调用脚本。

⑤ [操作] 在 `/etc/cron.d` 中集中管理系统任务

```
# /etc/cron.d/weekly-check
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
MAILTO=""

30 15 * * 3 monitor /usr/local/libexec/weekly-check \
>> /var/log/weekly-check.log 2>&1
```

建议:

- 使用简单文件名, 不依赖编辑器备份后缀;
- 文件由 `root` 管理且不得可被普通用户修改;
- 末尾保留换行;
- 脚本有明确 `shebang` 和执行权限;
- 不通过 `source ~/.bashrc` 把交互环境整体注入计划任务。

⑥ [边界] 周期目录与 `anacron` 只做必要辨识

`/etc/cron.hourly`、`/etc/cron.daily` 等目录由发行版提供的调度链调用。把脚本放进目录不代表立即执行, 也不等于你已经理解具体运行时间。应检查 `/etc/anacrontab`、`/etc/cron.d/0hourly` 或本机对应配置。

本章不完整展开 `anacron` 的 `period`、`delay` 和 `job identifier`; 需要精确控制用户、分钟和日志时, 直接使用用户 `crontab`、`/etc/cron.d` 或 `timer` 更容易验收。

⑦ [验证点] 对 cron 做四层验收

```
crontab -l -u monitor
systemctl is-active crond.service
systemctl is-enabled crond.service
journalctl -u crond.service --since today --no-pager
stat /var/log/weekly-check.log
```

对于 `/etc/cron.d` :

```
sed -n '1,120p' /etc/cron.d/weekly-check
stat -c '%U %G %a %n' /etc/cron.d/weekly-check
```

验证层次:

配置: 字段、用户、环境和命令正确
调度器: `crond` 当前运行, 并满足题目持久要求
触发: 日志中出现目标任务活动
执行: 目标用户能够运行, 错误被记录
功能: 业务文件或消息正确更新

[Cheatsheet] 用户表不写用户字段; 系统表必须写用户字段; 显式设置 PATH 和输出; `crontab -r` 删除整张表; `%` 要转义或移入脚本。

[诊断专题] 手工执行成功, 但 cron 失败

“我在终端里运行没问题”只证明当前用户、当前目录和当前环境能运行。cron 使用的用户、环境和输入输出路径可能完全不同。诊断应减少变量, 而不是反复等待下一个触发时间。

① [诊断] 先把症状分成三类

没有触发证据
→ 查任务是否装在正确用户、字段时间、`crond` 和时区

有触发证据但命令失败
→ 查身份、环境、路径、权限、`shebang`、`SELinux` 和依赖

命令似乎成功但业务产物不对
→ 查退出码设计、工作目录、目标路径、覆盖/追加和业务验证

② [诊断] 确认查的是正确用户和正确入口

```
crontab -l -u monitor
sudo grep -R --line-number --fixed-strings 'weekly-check' \
    /etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null
```

同一命令可能同时存在于 root crontab、目标用户 crontab、`/etc/cron.d` 和 timer。重复入口会造成重复执行；只看当前登录用户的 `crontab -l` 可能遗漏真正定义。

直接读取 spool 只用于调查，不用于修改。

③ [诊断] 用目标用户和精简环境复现

```
sudo -u monitor env -i \
    HOME=/home/monitor \
    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
    SHELL=/bin/bash \
    /usr/local/libexec/weekly-check
printf 'exit=%s\n' "$?"
```

这个测试比“切换成 monitor 后在完整登录环境运行”更接近 cron。若失败，下一条最有区分度的证据通常是脚本 stderr、文件权限或缺失命令，而不是继续修改时间字段。

④ [诊断] 按稳定顺序检查常见差异

```
用户/组
→ 绝对命令路径
→ shebang 与执行权限
→ PATH 和必要变量
→ 工作目录与相对文件
→ 目标目录写权限
→ SELinux AVC 证据
→ 网络、挂载、DNS、凭据等外部依赖
→ stdout/stderr 和退出码
```

不要把整个 `.bashrc` 或 `.profile` source 进 cron 来掩盖脚本依赖。需要的变量应在调度配置、脚本或受控环境文件中明确声明。

⑤ [诊断] 检查 %、重定向和邮件假设

- 命令中出现 `date +%F`、SQL `%` 或 URL 编码时，先检查未转义 `%`；
- 日志文件由 root 预先创建且目标用户不可写时，Shell 在执行命令前就会因重定向失败；
- `MAILTO` 已设置但没有 MTA 时，没有邮件不能证明任务没有运行；
- `>/dev/null 2>&1` 会丢失故障证据，不应作为排错初始状态。

⑥ [诊断] 修复后立即做可控验证

不必把正式周期改成每分钟并忘记恢复。优先：

- 1. 用精简环境直接运行脚本；
- 2. 临时创建一个明确标记、很快触发的测试入口；
- 3. 验证后删除测试入口；
- 4. 恢复并核对正式表达式；
- 5. 检查真实产物和时间戳。

[Cheatsheet] 正确用户 → 精简环境 → 绝对路径 → 权限/SELinux → 触发日志 → stdout/stderr → 真实产物；
不要用完整登录环境替代复现。

[知识专题] timer 和 service 是两个状态对象

systemd timer 的价值不只是另一种日期语法。它把“何时激活”和“怎样执行”拆成两个 unit，使 service 可以独立验证，并把退出状态纳入 systemd 和 journal。这个拆分也意味着排错时不能只查 timer。

① [知识点] 默认激活同名 service

```
weekly-check.timer
    ↓ 默认激活
weekly-check.service
```

也可用 `Unit=` 指定其他 unit。为了降低认知负担，通常让 timer 和 service 除后缀外同名。只需要 enable timer；被 timer 触发的 service 通常不应单独 enable，否则可能在启动链中额外运行。

若 timer 到点时目标 unit 已经 active，systemd 不会自动启动第二个实例，也不会重启它。重复 timer 不适合搭配长期保持 active 的 `RemainAfterExit=yes` oneshot service，否则可能只触发第一次。

② [知识点] 日历 timer 与单调 timer 的时间基准

属性	起点或时间基准	典型用途
<code>OnCalendar=</code>	实时时钟和日历	每天 02:15、每周三
<code>OnBootSec=</code>	系统启动时刻	开机 10 分钟后
<code>OnActiveSec=</code>	timer 自身被激活时刻	timer 启动后延迟一次
<code>OnStartupSec=</code>	service manager 启动时刻	用户 manager 等场景
<code>OnUnitActiveSec=</code>	目标 unit 上次被激活时刻	每次激活后再等一段时间
<code>OnUnitInactiveSec=</code>	目标 unit 上次变为 inactive 的时刻	任务完成后再等待固定间隔

多个表达式可以共存，任何一个到期都可触发目标 unit。`OnBootSec=` 或 `OnStartupSec=` 已经落在过去时，激活 timer 后可能立即到期；其他单调表达式和日历表达式应按对应规则判断。

③ [知识点] `Persistent=` 只补日历 timer 的遗漏

`Persistent=true` 记录 service 上次由 timer 触发的时间。timer 再次激活时，如果在 inactive 期间本应至少触发一次，就立即补一次。边界：

- 只对带 `OnCalendar=` 的 timer 有效；
- 多次遗漏不会逐次回放，而是形成一次补触发；
- 补触发仍受 `RandomizedDelaySec=` 影响；
- timer 从未启用或状态记录被清理时，不能凭想象推导历史；
- 仍要检查 service 是否成功和业务任务是否幂等。

④ [知识点] `RandomizedDelaySec=` 与 `AccuracySec=` 目的相反

`RandomizedDelaySec=5min` 在计算出的下一次时间上增加 0 到 5 分钟的随机延迟，用于避免多台主机同时产生负载峰值。

`AccuracySec=1min` 允许 systemd 在目标时间之后的一个精度窗口内，把本机多个 timer 的唤醒合并，以减少唤醒开销。它不是“最大允许业务延迟”的完整 SLA，也不是随机摊开主机的属性。

组合顺序可以理解为：

```
先计算基础触发时间
→ 加上 RandomizedDelaySec 的随机值
→ 再在 AccuracySec 窗口内做本机合并
```

需要尽可能精确地观察随机延迟分布时，可把 `AccuracySec=` 设得更小；不要无理由把所有 timer 都设为微秒级精度。

⑤ [知识点] timer 的 waiting 不证明 service 成功

`systemctl list-timers --all` 常见列：

- `NEXT`：预计下一次触发时间；
- `LEFT`：距下一次还有多久；
- `LAST`：最近一次触发时间；
- `PASSED`：距最近一次过去多久；
- `UNIT`：timer 名；
- `ACTIVATES`：目标 unit。

这些列证明调度时间和触发关系。service 的 `Result`、`ExecMainCode`、`ExecMainStatus`、journal 和产物才证明执行结果。

[Cheatsheet] timer 决定触发，service 决定执行；日历用 `OnCalendar`，相对启动或上次激活用单调属性；`Persistent` 只补日历遗漏；随机延迟和精度窗口不是一回事。

[操作专题] 建立、启用和验证持久 timer

稳定流程是 service-first：先确认脚本在目标用户下成功，再建立 oneshot service；service 独立成功后，才让 timer 定期激活它。这样能把脚本问题、service 配置问题和日历问题分层定位。

① [操作] 在目标身份和精简环境中验证工作负载

```
sudo -u reporter env -i \  
  HOME=/home/reporter \  
  PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \  
  /usr/local/libexec/daily-report  
printf 'exit=%s\n' "$?"  
stat /var/lib/daily-report/latest.txt
```

先修复脚本、目录权限和外部依赖。不要在脚本尚未成功时同时引入 service 与 timer。

② [操作] 创建并直接验证 oneshot service

```
# /etc/systemd/system/daily-report.service  
[Unit]  
Description=Generate Daily Operations Report  
  
[Service]  
Type=oneshot  
User=reporter  
Group=reporter  
WorkingDirectory=/var/lib/daily-report  
UMask=0027  
ExecStart=/usr/local/libexec/daily-report
```

静态检查和直接运行：

```
systemd-analyze verify /etc/systemd/system/daily-report.service  
systemctl daemon-reload  
systemctl start daily-report.service  
systemctl show daily-report.service \  
  -p Result,ExecMainCode,ExecMainStatus  
journalctl -u daily-report.service -b --no-pager  
stat /var/lib/daily-report/latest.txt
```

`Type=oneshot` 让 systemd 等待命令结束并记录退出状态。对周期触发的一次性脚本，通常不需要 `RemainAfterExit=yes`，否则 service 会保持 active，后续 timer 到期时不会再次启动它。

③ [操作] 创建日历 timer

```
# /etc/systemd/system/daily-report.timer
[Unit]
Description=Run Daily Operations Report

[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=5min
AccuracySec=30s
Unit=daily-report.service

[Install]
WantedBy=timers.target
```

`AccuracySec=30s` 在本任务中表达允许较小的本机合并窗口；它不是必须值。应根据题目是否要求严格时间和资源摊开做选择。

④ [验证点] 先解析表达式，再加载 unit

```
systemd-analyze calendar '*-*-* 02:15:00'
systemd-analyze verify \
    /etc/systemd/system/daily-report.service \
    /etc/systemd/system/daily-report.timer
systemctl daemon-reload
```

`systemd-analyze calendar` 会规范化表达式并计算后续触发时间。它证明表达式能解析以及日历含义，不证明 unit 已加载、timer 已启动或 service 能运行。

⑤ [操作] 启用并立即启动 timer

```
systemctl enable --now daily-report.timer
systemctl is-enabled daily-report.timer
systemctl is-active daily-report.timer
```

`enable --now` 同时建立持久启用链接并启动当前 timer。仍应分别查询 active 与 enabled，不用一个状态代替另一个。

不要 enable `daily-report.service`。service 由 timer 触发，并可在排错时手工 `start`。

⑥ [查询] 查询 NEXT/LAST 与 service 结果

```
systemctl list-timers --all | grep -F daily-report
systemctl status daily-report.timer --no-pager -l
systemctl show daily-report.timer \
    -p NextElapseUsecRealtime,LastTriggerUsec,AccuracyUsec,RandomizedDelayUsec

systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.timer -u daily-report.service \
    --since today --no-pager
stat /var/lib/daily-report/latest.txt
```

属性名称在 `systemctl show` 中通常使用微秒后缀，与 unit 文件中的 `...Sec=` 是同一设置的底层表现。考试核心是读懂状态，不需要记住所有内部 D-Bus 属性。

⑦ [边界] 验收补跑和随机延迟需要真实时间场景

静态检查无法证明：

- 主机关机错过时间后是否补触发；
- 随机延迟落在何处；
- suspend/resume 和日历 timer 的实际行为；
- 多台主机的负载是否真正摊开。

真实环境测试应记录关机前状态、错过的日历时间、开机时间、timer LAST、service journal 和产物更新时间。本章当前仅给出推荐验证方法，尚未完成这些实机验证。

[Cheatsheet] 精简环境跑脚本 → 创建 service → `verify` → 手工启动 service → 创建 timer → `calendar` → `enable --now` → NEXT/LAST → service Result → journal 和产物。

[操作专题] 单调 timer、transient timer 与安全迁移

日历 timer 回答“钟表到某个时间”；单调 timer 回答“从某个事件起过多久”。`systemd-run` 还能临时创建 timer/service 对，适合验证命令或做短期任务，但不能替代要求重启后仍存在的持久 unit。

① [操作] 使用 `OnBootSec=` 和 `OnUnitActiveSec=`

```
[Timer]
OnBootSec=10min
OnUnitActiveSec=30min
AccuracySec=1min
```

这表示开机约 10 分钟后第一次触发，之后相对于目标 unit 每次激活时间每 30 分钟再次触发。若任务运行时间很长，`OnUnitActiveSec=` 的下一次基准从激活开始；需要从任务结束后再等待，应考虑

OnUnitInactiveSec=。

单调 timer 不受本地时区解释影响。休眠期间是否推进取决于使用的时钟和 `WakeSystem=`；这不是 RHCSA 主操作，本章只保留边界。

② [操作] 创建一次 transient timer

```
systemd-run \  
  --unit=health-test \  
  --on-active=2min \  
  --timer-property=AccuracySec=1s \  
  /usr/local/libexec/health-test
```

系统通常会创建：

```
health-test.timer  
health-test.service
```

查询：

```
systemctl status health-test.timer --no-pager  
systemctl list-timers --all | grep -F health-test  
journalctl -u health-test.timer -u health-test.service -b --no-pager  
systemctl show health-test.service -p Result,ExecMainStatus
```

transient unit 没有写入 `/etc/systemd/system` 的持久文件，manager 或系统重启后通常不应被当作仍存在。具体回收时点还与 unit 状态和收集策略有关。

③ [操作] 创建临时日历 timer

```
systemd-run \  
  --unit=report-preview \  
  --on-calendar='*-*-* 23:40:00' \  
  /usr/local/libexec/daily-report --preview
```

它适合观察 systemd 生成的 timer/service、journal 和命令运行方式。题目要求持久 timer 时，仍必须编写 `.service` 与 `.timer` 文件，并执行 `daemon-reload`、`enable` 和分层验证。

④ [知识点] transient timer 的边界

- `systemd-run` 创建的 service 默认执行环境与交互 Shell 不同；
- 使用 timer 选项时，立即启动的是 timer，service 到期后才触发；
- `--unit=` 让名称可预测，便于查询和清理；
- 不要把命令参数中的 `$` 展开时机想当然，Shell 展开和 systemd manager 展开是不同层；

- transient 成功不证明对应的持久 unit 文件、用户、权限和 enable 状态正确。

⑤ [操作] 从 cron 迁移到 timer 的安全顺序

```
记录旧 cron 定义与当前产物
→ 确认执行用户、环境、输出和并发语义
→ 目标用户精简环境运行脚本
→ 创建 service 并独立验收
→ 创建 timer 并解析日历
→ 启用 timer，观察至少一次可控触发
→ 检查 service Result、journal 和产物
→ 删除旧 cron 入口
→ 全局搜索重复调度
→ 最终回归
```

不要先删旧任务再开始调试新 service；也不要让旧 cron 和新 timer 长期并行，否则可能重复写入或并发冲突。

⑥ [验证点] 证明只剩一个调度入口

```
crontab -l -u reporter 2>/dev/null || true
grep -R --line-number --fixed-strings 'daily-report' \
    /etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null || true
systemctl list-timers --all | grep -F daily-report
systemctl cat daily-report.timer daily-report.service
```

“旧文件已经删除”不够，还要检查其他用户 crontab、`/etc/cron.d` 和同名或不同名 timer。

[Cheatsheet] 相对启动用 `OnBootSec`，从上次激活计时用 `OnUnitActiveSec`；`systemd-run` 只做临时 timer；迁移先证明新链路，再删除旧入口，最后查重。

[诊断专题] timer 在等待，但任务没有正确完成

timer 故障应分成“timer 没有被正确加载”“时间没有按预期到期”“发生触发但 service 失败”“service 成功但业务结果错误”。每一层使用不同证据。

① [诊断] timer 不在 `list-timers` 中

```
systemctl status daily-report.timer --no-pager -l
systemctl cat daily-report.timer
systemd-analyze verify /etc/systemd/system/daily-report.timer
systemctl daemon-reload
systemctl start daily-report.timer
```


重点检查：文件名和 unit 名、[Timer] 段、加载错误、是否忘记 `daemon-reload`、是否启动 timer。
`systemctl cat` 显示 `systemd` 实际合成的定义，不只看编辑器里的文件。

② [诊断] timer 为 waiting, 但 NEXT 不符合预期

```
systemd-analyze calendar '*--* 02:15:00'
systemctl list-timers --all | grep -F daily-report
systemctl show daily-report.timer \
    -p NextElapseUsecRealtime,AccuracyUsec,RandomizedDelayUsec
```

继续核对系统时区与当前时间，但完整时区和 `chrony` 诊断归第 14 章。注意 `RandomizedDelaySec=` 和 `AccuracySec=` 都可能让实际触发晚于基础日历时间。

③ [诊断] timer 有 LAST, 但 service failed

```
systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
systemctl status daily-report.service --no-pager -l
journalctl -u daily-report.service --since today --no-pager
systemctl cat daily-report.service
```

下一条证据通常来自 service：目标用户、工作目录、环境、`ExecStart`、文件权限、SELinux 和外部依赖。
不要继续只改 timer 日历。

④ [诊断] service success, 但没有正确产物

`Result=success` 只表示 `systemd` 认可主命令的退出状态。脚本可能捕获错误后仍退出 0，或者把文件写到了错误目录。

```
stat /var/lib/daily-report/latest.txt
head -n 20 /var/lib/daily-report/latest.txt
namei -l /var/lib/daily-report/latest.txt
```

检查脚本是否正确传播错误码，目标文件所有者、大小、内容和更新时间是否符合业务要求。

⑤ [诊断] `Persistent=true` 没有按预期补跑

依次确认：

1. timer 是否使用 `OnCalendar=`；
2. timer 是否在错过期间 `inactive`；
3. 重新激活 timer 时是否确实至少错过一次日历事件；
4. `RandomizedDelaySec=` 是否仍在等待；
5. service 是否已被触发但失败；
6. 状态时间戳是否被清理或 unit 是否换名。

静态阅读配置不能替代关机/开机场景测试。

⑥ [诊断] 任务重复或并发

```
grep -R --line-number --fixed-strings 'daily-report' \  
    /etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null || true  
systemctl list-timers --all | grep -F report  
pgrep -af '/usr/local/libexec/daily-report'
```

可能原因：旧 cron 未删除、存在两个 timer、手工启动与定时触发重叠，或任务运行时间超过周期。是否允许并发属于工作负载设计；可在脚本或 service 中使用明确锁机制，但不在本章展开复杂锁实现。

[Cheatsheet] 不在列表查加载；NEXT 错查日历、时区、随机与精度；有 LAST 查 service；service 成功再查产物；补跑和随机效果必须 live test。

[经典任务] 配置带明确用户和证据链的周期健康检查

环境

系统已有用户 `monitor` 和脚本 `/usr/local/libexec/web-health`。脚本读取本机服务状态，并把最终报告写入 `/var/lib/web-health/latest.txt`。root 手工执行脚本成功，但目前没有可靠周期任务。

现状：

- `/var/lib/web-health` 已存在，所有者当前为 `root:root`；
- `/var/log/web-health-cron.log` 尚不存在；
- `monitor` 的 crontab 可能为空；
- 不假设本机邮件系统可用；
- 不允许通过 `chmod 777` 解决权限。

目标终态

1. 每周三 15:30 以 `monitor` 用户执行；
2. 使用 `monitor` 的用户 crontab，不使用 root crontab 或 `/etc/cron.d`；
3. crontab 显式定义 `SHELL=/bin/bash`、受控 `PATH` 与 `MAILTO=""`；
4. stdout/stderr 追加到 `/var/log/web-health-cron.log`；
5. `monitor` 对报告目录和日志文件只获得完成任务所需的权限；
6. 任务正文使用绝对路径，不把复杂业务逻辑直接写进 crontab；
7. 验收必须证明：目标用户精简环境可运行、crontab 字段正确、crond 当前运行并满足持久要求、输出和报告文件正确更新。

限制条件

- 不删除其他用户或其他现有 cron 任务；
- 不把用户字段写进用户 crontab；
- 不把“cron active”扩大为任务成功；
- 不用 `>/dev/null 2>&1` 隐藏错误；
- 不编造等待到正式周三 15:30 的执行结果，可设计可撤销的临时验证。

验收证据

用户与目录权限
→ 精简环境退出码
→ monitor 的 crontab 原文
→ cron active/enabled
→ 测试触发或直接脚本证据
→ cron 日志
→ 报告文件所有者、大小和更新时间

[经典任务] 将需补执行的 cron 任务迁移为 systemd timer

环境

`/etc/cron.d/daily-report` 当前每天 02:15 以 `reporter` 用户执行：

```
15 2 * * * reporter /usr/local/libexec/daily-report \  
>> /var/log/daily-report-cron.log 2>&1
```

脚本应生成 `/var/lib/daily-report/latest.txt`。服务器可能在 02:15 关机，开机后必须补执行一次。多台服务器不应在完全相同的时刻开始任务，且管理员希望从 systemd 获取明确退出状态和 journal。

目标终态

1. 建立 `daily-report.service`，`Type=oneshot`，以 `reporter` 用户和组执行；
2. 明确工作目录、umask 和绝对 `ExecStart`；
3. 建立同名 `daily-report.timer`，每天 02:15 触发；
4. 配置 `Persistent=true`；
5. 配置最多 5 分钟随机延迟和可解释的 `AccuracySec=`；
6. service 可独立启动，退出状态成功并生成非空报告；
7. timer 当前 active、持久 enabled，并能显示 NEXT/LAST；
8. journal 能区分 timer 触发和 service 执行；
9. 新链路验收后删除旧 `/etc/cron.d/daily-report`，并证明没有重复调度入口。

限制条件

- 不 enable service，只 enable timer；
- 不在 timer 中写 `ExecStart=`；
- 不先删除旧 cron 再调试新 service；
- 不把 timer active 解释为 service 已成功；
- 不声称已经完成关机错过后的 live 补跑测试；
- 不修改其他 cron 或 timer。

验收证据

```
旧 cron 基线
→ reporter 精简环境
→ service 静态检查与独立执行
→ timer 日历解析与静态检查
→ active/enabled
→ NEXT/LAST
→ service Result/ExecMainStatus
→ journal
→ 报告文件
→ 旧入口删除与全局查重
```

[参考解答] 周期健康检查：先修正身份与文件权限，再安装 crontab

参考解答展示调查顺序和证据链，不代表已在本会话的 RHEL 9 VM 中执行。

① [调查] 保存基线并检查对象

```
id monitor
getent passwd monitor
ls -ld /var/lib/web-health /var/log
ls -l /usr/local/libexec/web-health
head -n 1 /usr/local/libexec/web-health
crontab -l -u monitor > /root/monitor.crontab.before 2>/dev/null || :
systemctl is-active crond.service
systemctl is-enabled crond.service
```

确认脚本确实存在且有 shebang。不要覆盖已有 crontab；编辑时应保留现有条目。

② [操作] 只授予需要的目录与日志权限

一种可审阅的处理方式：

```
install -d -o monitor -g monitor -m 0750 /var/lib/web-health
install -o monitor -g monitor -m 0640 /dev/null \
/var/log/web-health-cron.log
```

若目录内已有其他业务文件，不能直接递归 `chown`；应先调查具体所有权需求，再做最小修改。

③ [验证] 在精简环境直接运行脚本

```
sudo -u monitor env -i \
HOME=/home/monitor \
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
SHELL=/bin/bash \
/usr/local/libexec/web-health \
>> /var/log/web-health-cron.log 2>&1
printf 'exit=%s\n' "$?"

stat -c '%U %G %a %s %y %n' \
/var/log/web-health-cron.log \
/var/lib/web-health/latest.txt
```

失败时先读日志并修复脚本依赖，不要继续安装 `cron`。

④ [操作] 编辑 `monitor` 的用户 `crontab`

```
crontab -u monitor -e
```

保留原有内容并增加：

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
MAILTO=""

30 15 * * 3 /usr/local/libexec/web-health >> /var/log/web-health-cron.log 2>&1
```

这里没有用户字段，因为这是 `monitor` 自己的 `crontab`。

⑤ [静态核对] 检查字段、环境和调度器

```
crontab -l -u monitor
systemctl is-active crond.service
systemctl is-enabled crond.service
journalctl -u crond.service -b --no-pager | tail -n 50
```

若题目要求重启后调度器仍工作而 `crond` 未 `enabled`，应按第 12 章的 `service` 状态方法处理；本章不重讲完整 `enable` 语义。

⑥ [验证] 设计可撤销的短期测试

不能等待正式周三时，可临时把同一个脚本放入一条最近分钟的测试规则，并加唯一标识或测试参数。测试完成后立即删除测试行，重新核对正式规则。

更安全的最低证据是：

- 精简环境脚本已经成功；
- crontab 内容和字段静态正确；
- crond active；
- stdout/stderr 目标可写；
- 报告产物已由 `monitor` 正确生成。

没有真实等待触发时，应在 QA 中标记“正式周期触发未 live test”，不能声称已跑通。

⑦ [最终验收]

```
id monitor
crontab -l -u monitor
systemctl is-active crond.service
systemctl is-enabled crond.service
stat -c '%U %G %a %s %y %n' \
    /var/log/web-health-cron.log \
    /var/lib/web-health/latest.txt
```

典型错误：

- 在用户 crontab 中写 `monitor` 用户字段；
- root 创建日志文件后不给 `monitor` 写权限；
- 使用相对脚本或命令路径；
- 用 `crontab -r` 删除整张表；
- 临时测试后忘记恢复正式规则；
- 仅凭 `crond active` 宣布完成。

[参考解答] cron 迁移 timer：service-first、验证后切换、最

后查重

① [调查] 保存旧入口与产物基线

```
cp -a /etc/cron.d/daily-report \
    /root/daily-report.cron.before
sed -n '1,120p' /etc/cron.d/daily-report
id reporter
ls -l /usr/local/libexec/daily-report
stat /var/lib/daily-report/latest.txt 2>/dev/null || true
systemctl list-timers --all | grep -F daily-report || true
```

确认旧任务的时间、用户、命令、输出路径和任何环境变量。若脚本依赖旧 cron 里的 PATH，应在 service 或脚本中显式处理。

② [验证] 用目标用户和精简环境运行脚本

```
install -d -o reporter -g reporter -m 0750 /var/lib/daily-report

sudo -u reporter env -i \
    HOME=/home/reporter \
    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
    /usr/local/libexec/daily-report
printf 'exit=%s\n' "$?"
stat -c '%U %G %a %s %y %n' /var/lib/daily-report/latest.txt
```

脚本失败时先修复其路径、权限、SELinux 或外部依赖。

③ [操作] 创建 oneshot service

```
# /etc/systemd/system/daily-report.service
[Unit]
Description=Generate Daily Operations Report

[Service]
Type=oneshot
User=reporter
Group=reporter
WorkingDirectory=/var/lib/daily-report
UMask=0027
ExecStart=/usr/local/libexec/daily-report
```

静态检查并直接运行：

```
systemd-analyze verify /etc/systemd/system/daily-report.service
systemctl daemon-reload
systemctl start daily-report.service
systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.service -b --no-pager
stat /var/lib/daily-report/latest.txt
```

期望 `Result=success` 且 `ExecMainStatus=0`，同时报告文件非空并由正确用户更新。不要只看 `systemctl start` 返回 0。

④ [操作] 创建 timer

```
# /etc/systemd/system/daily-report.timer
[Unit]
Description=Run Daily Operations Report

[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=5min
AccuracySec=30s
Unit=daily-report.service

[Install]
WantedBy=timers.target
```

⑤ [静态核对] 解析日历并检查两个 unit

```
systemd-analyze calendar '*-*-* 02:15:00'
systemd-analyze verify \
    /etc/systemd/system/daily-report.service \
    /etc/systemd/system/daily-report.timer
systemctl daemon-reload
systemctl cat daily-report.service daily-report.timer
```

`calendar` 输出的下一次时间是静态日历证据；随机延迟和精度窗口还会影响实际调度。

⑥ [操作] 启用 timer，不 enable service

```
systemctl enable --now daily-report.timer
systemctl is-enabled daily-report.timer
systemctl is-active daily-report.timer
systemctl list-timers --all | grep -F daily-report
```


⑦ [验证] 检查触发关系与执行结果

```
systemctl status daily-report.timer --no-pager -l
systemctl show daily-report.service \
  -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.timer -u daily-report.service \
  --since today --no-pager
stat -c '%U %G %a %s %y %n' /var/lib/daily-report/latest.txt
```

若尚未到正式时间，service 的直接成功、timer 的 NEXT、active/enabled 和静态配置只构成阶段性证据；正式日历触发、随机延迟及关机补跑必须后续 live test。

⑧ [切换] 新链路通过后删除旧 cron

```
rm -f /etc/cron.d/daily-report
systemctl try-restart crond.service
```

通常 crond 会自动发现文件变化；try-restart 是否必要应按现场策略决定。不要无理由重启整个系统。

⑨ [最终验收] 搜索重复入口

```
crontab -l -u reporter 2>/dev/null || true
grep -R --line-number --fixed-strings 'daily-report' \
  /etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null || true

systemctl is-active daily-report.timer
systemctl is-enabled daily-report.timer
systemctl list-timers --all | grep -F daily-report
systemctl show daily-report.service \
  -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.timer -u daily-report.service \
  --since today --no-pager
stat /var/lib/daily-report/latest.txt
```

典型错误：

- 在 service 未独立成功前就启用 timer；
- enable service 导致额外启动；
- 配置 Persistent=true 却使用纯单调 timer；
- 只查 timer，不查 service Result；
- 旧 cron 和新 timer 同时保留；
- 将随机延迟理解为固定 5 分钟；
- 将 AccuracySec 理解为随机摊开；
- 未做关机补跑 live test 却声称已验证。

[本章收束] 让调度器、执行身份和结果证据对齐

工作方法：先静态、再触发、最后业务

三种调度器共享同一管理逻辑：

```
at: job → atd → 保存的脚本与环境 → 输出/产物
cron: entry → crond → 用户与精简环境 → 输出/产物
timer: timer → service → Result/journal → 输出/产物
```

选择时先问任务生命周期。提交或安装后，先验证静态定义；触发前确认调度器；触发后转向工作负载退出状态；最后以真实产物完成验收。

调度选择矩阵

问题	选择或下一步
只运行一次	<code>at</code> ，提交后用 <code>atq</code> 和 <code>at -c</code> 审核
简单固定周期，题目明确要求 <code>crontab</code>	用户 <code>crontab</code> 或 <code>/etc/cron.d</code>
需要明确系统用户字段	<code>/etc/cron.d</code> 或 <code>systemd service</code> 的 <code>User=</code>
需要关机错过后补一次	<code>OnCalendar=</code> + <code>Persistent=true</code>
需要开机后延迟	<code>OnBootSec=</code>
需要从上上次激活后计算间隔	<code>OnUnitActiveSec=</code>
需要多台主机摊开负载	<code>RandomizedDelaySec=</code>
需要观察 <code>systemd</code> 临时调度	<code>systemd-run --on-active</code> 或 <code>--on-calendar</code>
手工成功但计划失败	目标用户 + <code>env -i</code> 重建精简环境
<code>timer waiting</code> 但无报告	查 <code>service Result</code> 、 <code>journal</code> 和产物，不继续只看 <code>timer</code>

帮助入口

```
man at
man atd
man crontab
man 5 crontab
man cron
man systemd.timer
man systemd.time
man systemd-run
systemd-analyze calendar --help
systemctl list-timers --help
```

向下一章交接

本章已经把“任务何时运行”转换成了可验证的调度对象，并反复区分配置、等待、触发、退出和业务产物。下一章《RPM 包、文件归属与软件事务》将转向系统内容的来源与安装状态：当 `at`、`crontab` 或某个辅助命令不存在时，应先确认软件包、文件归属和事务状态，而不是在本章无边界地展开软件安装。